CloudScale Lab

Project Progress Report — Completed Work and Remaining Tasks

Project title: SLA-Adaptive Kubernetes Autoscaling for Cloud Applications

Report date: 11 October 2026

Purpose: Summarize implementation completed so far and clearly separate verified deployment progress from work that is still outstanding.

1. Executive summary

The project has progressed from a local prototype to a real Kubernetes workload and controller deployment on an AWS EC2 instance running Ubuntu and K3s. The FastAPI workload is running and responding to health checks. The adaptive controller container is also running, but its logs show that it cannot yet retrieve Prometheus latency metrics and that the expected HPA named **webapp-hpa** has not been created. Therefore, the end-to-end adaptive autoscaling loop is **not yet operational**, and no live comparative experiment results have been established.

2. Work completed

Area	Verified progress
AWS / Kubernetes environment	An Ubuntu EC2 instance is provisioned in the Sydney AWS region. K3s is installed and the node was previously confirmed Ready.
Containerized workload	The local workload image was transferred to EC2 and imported into K3s containerd. The workload Deployment and ClusterIP Service were applied in namespace **sla-autoscaling**.
Workload health	The web application pod was observed as **1/1 Running**. The /health endpoint returned a successful JSON response, and pod resource metrics were available through kubectl top pods.
Controller image	The controller image archive was split into chunks, transferred, reassembled, and imported into K3s. The image is available as docker.io/library/cloudscale-controller:local for linux/amd64.
Controller RBAC and deployment	The service account, Role, RoleBinding, ConfigMap, Lease, and controller Deployment were applied. The controller pod initially failed because the deployment contained the literal ${CONTROLLER_IMAGE}; the image was then updated to the imported local image.
Controller runtime	The controller pod subsequently reached **1/1 Running**, acquired the Kubernetes Lease, and started its live-mode control loop with dry_run=True. This means it is not yet making live HPA target changes.
Research design	The planned comparison is conventional HPA with a fixed 70% CPU target versus latency-adaptive target adjustment. The target SLO is 200 ms P95 latency; the controller uses a 10-second loop and 60-second stabilization/cooldown, with downward target adjustments of 5–7.5 percentage points and upward adjustments of 1 point, bounded between 40% and 85%.

3. Current known blockers

Blocker	Details
Prometheus unavailable	Controller logs show DNS resolution failure when querying the configured Prometheus endpoint. Prometheus has not yet been confirmed installed and reachable in the cluster.
HPA missing	Controller logs report a 404 because the HPA resource webapp-hpa does not exist in namespace sla-autoscaling.
Dry-run mode	The controller logs explicitly show dry_run=True. Live HPA patching has not been demonstrated.
Experimental evidence not collected	No paired live Locust runs, telemetry exports, or validated live comparisons between fixed and adaptive HPA have been completed.

4. Work remaining

A. Deploy and configure Prometheus

Install a lightweight Prometheus setup suitable for the single EC2/K3s node, expose it internally via a Kubernetes Service, and configure scraping of the workload /metrics endpoint at the intended 5-second interval. Confirm the P95 PromQL query returns real values.

B. Create the HPA resources

Apply the conventional fixed-target HPA and the adaptive HPA configuration as required by the experiment. Ensure the resource name and namespace match the controller configuration (webapp-hpa in sla-autoscaling) and confirm metrics-server / CPU metrics are available.

C. Connect and verify the controller

Correct the Prometheus URL/configuration if needed. Verify the controller can query P95 latency and read the HPA target. Keep dry-run enabled while checking decisions; only disable dry-run after confirming metrics, permissions, and safe behavior.

D. Run paired load tests

Use Locust to run identical workload profiles for fixed and adaptive configurations. At minimum, run ramp and spike profiles; add sawtooth if time and resources allow. Keep workload duration, traffic shape, and settings comparable.

E. Capture and analyze results

Export raw telemetry and test results to CSV/JSON. Calculate P95 latency, SLO violation rate (P95 above 200 ms), scaling delay, average/peak replicas, and controller oscillation. Do not treat the earlier local simulator output as live cluster evidence.

F. Finish report and demo

Prepare architecture and methodology diagrams, describe the environment and test parameters, present measured results and limitations, include screenshots/logs, and connect the project UI to live metrics only if time permits.

Optional: Minecraft workload

The blueprint mentions a stateful Minecraft workload. This is optional for the minimum web-microservice evaluation and should be deferred until the primary autoscaling experiment works.

5. Recommended execution order

• Install Prometheus and confirm the application metrics are being scraped.

• Create the HPA and verify the controller is configured for the same HPA name and namespace.

• Verify adaptive decisions in dry-run mode, then carefully enable live HPA updates.

• Run the fixed 70% baseline and adaptive tests under comparable Locust load.

• Export raw data, create graphs, and write conclusions based only on measured results.

6. Research logic reference

The intended controller uses a 200 ms P95 latency SLO. If P95 is at least 300 ms, lower the CPU target by 7.5 percentage points (minimum 40%); if P95 is at least 240 ms but below 300 ms, lower it by 5 points (minimum 40%); if P95 is 160–239 ms, hold the target; if P95 is below 160 ms, raise it by 1 point (maximum 85%). The loop is intended to run every 10 seconds, with a 60-second cooldown/stabilization period. These are design parameters, not a claim that the full control loop is already working live.

7. Completion status

Implemented: EC2/K3s foundation, workload deployment, controller image import, RBAC resources, and a running controller pod.
In progress: Prometheus connectivity and HPA setup.

Not yet demonstrated: active adaptive HPA modification, paired live load tests, empirical comparison dataset, final results and live dashboard.

Accuracy note: This report records the state evidenced by the project notes and terminal outputs available at report creation. It does not claim Prometheus, HPA-based autoscaling, Grafana, Locust benchmarking, or Minecraft deployment are complete.